

JUnit x TestNG

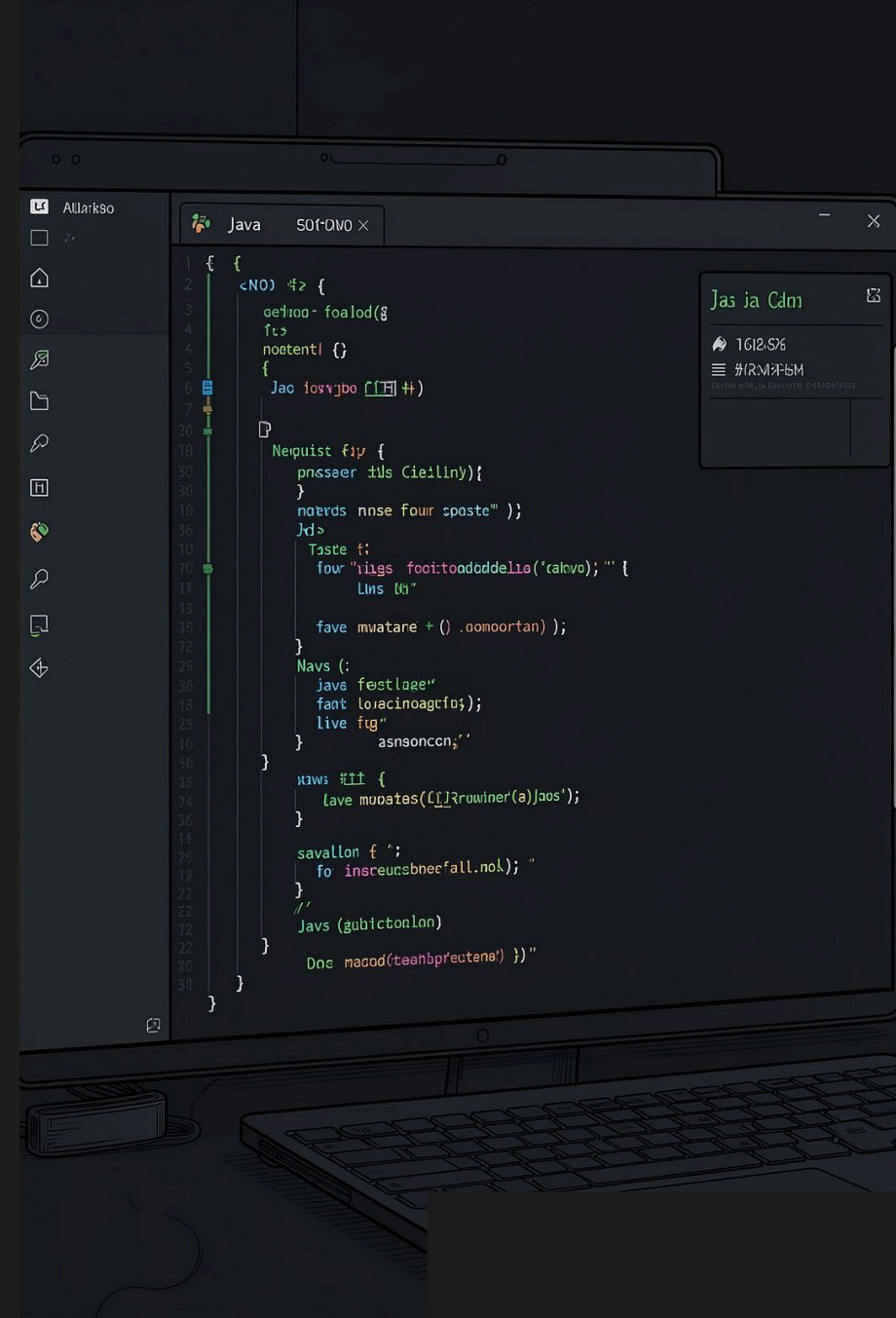
Testes Unitários em Java

Comparação prática utilizando uma Calculadora

JAVA

JUNIT

TESTNG



Testes Unitários

Um teste unitário verifica uma pequena parte do código — normalmente um método — para conferir se ela apresenta o comportamento esperado.

Cenário de sucesso

Entrada: 50 + 10

Esperado: **60** · Obtido: **60**

✓ Teste passou

Cenário de falha

Entrada: 50 + 10

Esperado: **60** · Obtido: **61**

✗ Teste falhou

📌 Framework: estrutura com códigos e ferramentas prontas que facilita a criação e execução dos testes.

JUnit

Framework simples e muito utilizado em projetos Java.

TestNG

Framework com recursos adicionais de organização e controle da execução.

Nossa Calculadora

Antes de testar, precisamos do código a ser verificado. A classe possui quatro métodos: `somar()`, `subtrair()`, `multiplicar()` e `dividir()`.

Método `somar()`

```
public int somar(int a, int b) {  
    return a + b;  
}
```

- `public int` → retorna um número inteiro
- `somar` → nome do método
- `int a, int b` → valores recebidos
- `return a + b` → realiza a soma

Método `dividir()`

```
public int dividir(int a, int b) {  
    if (b == 0) {  
        throw new IllegalArgumentException(  
            "Não é possível dividir por zero"  
        );  
    }  
    return a / b;  
}
```

- `if (b == 0)` → verifica divisor zero
- `throw` → lança uma exceção
- `IllegalArgumentException` → argumento inválido
- `return a / b` → divisão normal

📌 A divisão por zero é importante: vamos testar situações inválidas também.

Testando com JUnit

Antes de escrever os testes, precisamos preparar o objeto `Calculadora` que será utilizado.

```
private Calculadora calculadora;  
  
@BeforeEach  
void prepararCalculadora() {  
    calculadora = new Calculadora();  
}
```

`private Calculadora calculadora`

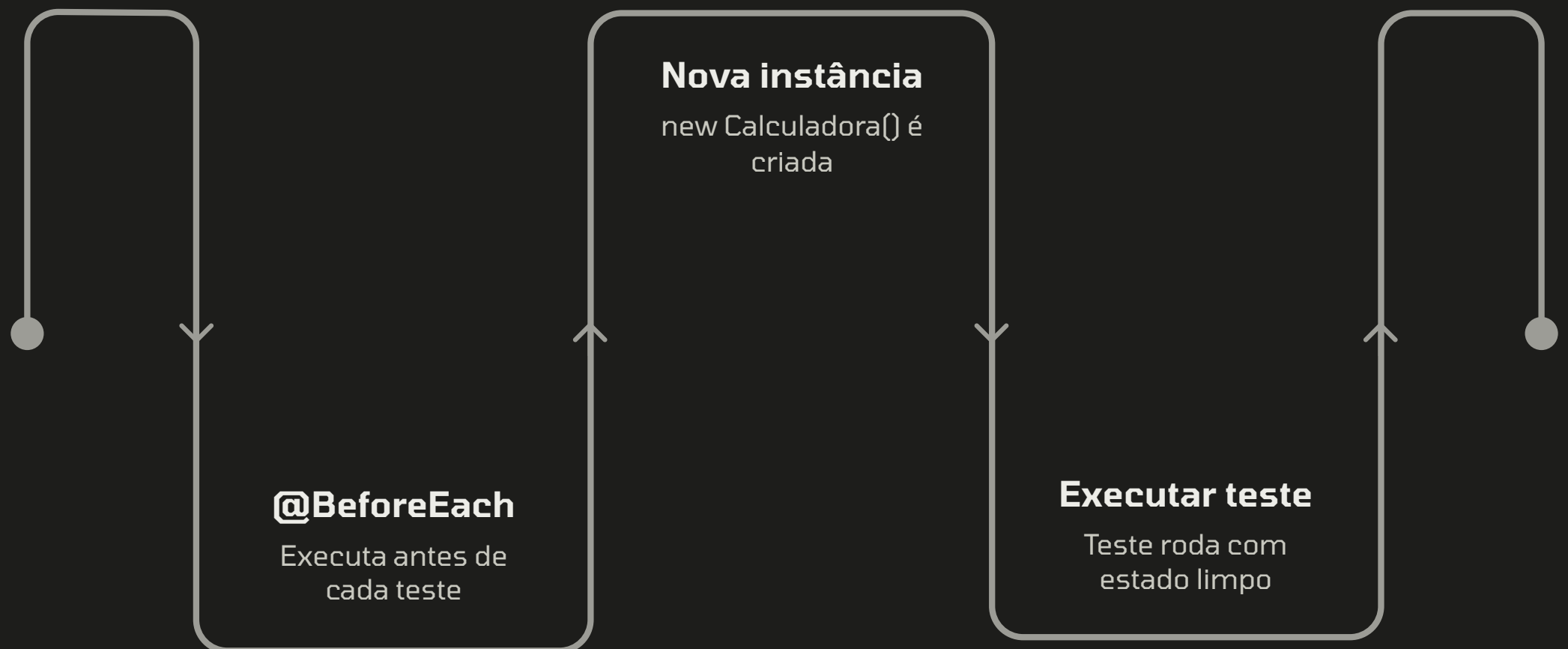
Variável que armazena o objeto utilizado nos testes.

`@BeforeEach`

Executa a preparação **antes de cada teste**.

`new Calculadora()`

Cria uma nova instância para cada teste, mantendo-os **independentes**.



JUnit — Verificando a Soma

```
@Test
void deveSomarDoisNumeros() {
    int resultado = calculadora.somar(50, 10);
    assertEquals(60, resultado);
}
```

1

@Test

Informa ao JUnit que este método deve ser executado como um teste.

2

calculadora.somar(50, 10)

Executa o método `somar` com os valores 50 e 10. O retorno é armazenado em `resultado`.

3

assertEquals(60, resultado)

Compara o valor esperado (60) com o valor retornado. `assert` = verificar.

✅ Esperado: 60 · Obtido: 60 → ✓ Teste passou

O Mesmo Teste com TestNG

Preparação

```
@BeforeMethod
void prepararCalculadora() {
    calculadora = new Calculadora();
}
```

@BeforeMethod → executa a preparação antes de cada teste, de forma semelhante ao @BeforeEach do JUnit.

Teste de soma

```
@Test
public void deveSomarDoisNumeros() {
    int resultado = calculadora.somar(50, 10);
    Assert.assertEquals(resultado, 60);
}
```

- @Test → identifica o método como teste
- calculadora.somar(50, 10) → mesma operação do JUnit
- Assert.assertEquals(resultado, 60) → verifica o resultado

✅ Resultado: 60 · Esperado: 60 → ✓ Teste passou

JUnit

```
assertEquals(60, resultado);
// esperado → resultado
```

TestNG

```
Assert.assertEquals(resultado, 60);
// resultado → esperado
```

A diferença está na **ordem dos parâmetros**. Nos testes simples, os dois frameworks realizam verificações muito semelhantes.

Testando uma Situação Inválida

Quando o divisor é zero, a Calculadora **lança uma exceção** em vez de retornar um número. O teste deve verificar esse comportamento.

JUnit

```
@Test
void deveImpedirDivisaoPorZero() {
    assertThrows(
        IllegalArgumentException.class,
        () -> calculadora.dividir(10, 0)
    );
}
```

- `assertThrows()` → verifica se a exceção é lançada
- `IllegalArgumentException.class` → exceção esperada
- `() ->` → expressão lambda do Java

TestNG

```
@Test(expectedExceptions =
    IllegalArgumentException.class)
public void deveImpedirDivisaoPorZero() {
    calculadora.dividir(10, 0);
}
```

- `expectedExceptions` → declara a exceção no `@Test`
- `calculadora.dividir(10, 0)` → executa a operação inválida
- Se a exceção acontecer → ✓ teste passa

📌 **Principal diferença:** JUnit usa `assertThrows()` dentro do método. TestNG declara a exceção esperada diretamente na anotação `@Test`.

JUnit x TestNG

JUnit

- Preparação: `@BeforeEach`
- Identificação: `@Test`
- Comparação: `assertEquals(esperado, resultado)`
- Exceção: `assertThrows()`

Simples, direto e muito utilizado em projetos Java. Controles mais avançados de execução podem ser menos diretos.

TestNG

- Preparação: `@BeforeMethod`
- Identificação: `@Test`
- Comparação: `Assert.assertEquals(resultado, esperado)`
- Exceção: `expectedExceptions` no `@Test`

Flexível, com suporte a grupos, prioridades, dependências e execução paralela. A quantidade maior de recursos pode aumentar a complexidade em projetos simples.

📄 Na nossa Calculadora, os dois frameworks conseguiram testar os mesmos comportamentos.

Conclusão

Com os testes implementados, verificamos automaticamente se os métodos da Calculadora apresentam os resultados e comportamentos esperados.

JUnit

Mais simples e direto no nosso exemplo. Ideal para projetos que precisam de uma abordagem objetiva.

TestNG

Oferece maior flexibilidade e recursos de controle de execução. Adequado para projetos com necessidades mais complexas.

Testes Unitários

Ajudam a identificar erros e evitar que alterações no código quebrem comportamentos já testados.

- ❏ Não existe um framework melhor em todas as situações. A escolha depende das necessidades do projeto. Os testes verificam os cenários definidos, mas não garantem que todo o sistema esteja livre de erros.